ENGINEERING VERIFICATION PASS

AURUM: The Seven Flaws and Their Fixes

A Deep Research and Design Review of the AURUM Decider Build Plan - Leakage, Labels, Cascade, Loop Realignment, Cross-Asset State, and Walk-Forward Compute

Prepared for: tskdev (Kaggle)
Specification Delta v1.0 to v1.1 - Pre-Phase-0 Gate Review
November 7, 2026

Project AURUM · DeerFlow Deep-Research Methodology · Model Judges, Code Executes

Table of Contents

Note: This Table of Contents is generated via field codes. To refresh page numbers after editing, right-click the TOC and select Update Field.

1. Executive Summary and Verdict Table....1
2. Mandate, Method, and Evidence Base....2
3. Flaw 1: Temporal and Price-Level Leakage in State Serialization....2
3.1 Confirmation: a price level is a date stamp....3
3.2 Feature-by-feature audit of the Section 7.1 fragment....3
3.3 Fix part one: the stationarity contract as an engineering rule....4
3.4 Fix part two: the three-layer automated leakage audit....5
4. Flaw 2: The Counterfactual Pair Paradox....5
4.1 Confirmation: the label cannot follow the flip....5
4.2 Fix: matched cohorts, bootstrap soft labels, and a demoted counterfactual metric....6
4.3 Dataset budget update....6
5. Flaw 3: Circular Dependency in the Six-Task Cascade....7
5.1 Confirmation: three defects in one design....7
5.2 The fix: a barrier-matrix world model with a deterministic decision layer....7
5.3 Exits: derived rules first, a management pass second....8
5.4 Calibration, the property that makes EV arithmetic meaningful....8
6. Flaw 4: Single-Path Hindsight Noise in Exit and Stop Labels....9
6.1 Confirmation: hindsight-argmax is a biased, noisy estimator....9
6.2 The fix: train on outcome distributions, decide on expectations....9
6.3 The bias, measured....9
7. Flaw 5: Looped Pretrained Latent Space Mismatch, Layers 13-16....10
7.1 Confirmation: the residual stream is depth-coded, and looping violates the contract....10
7.2 The realignment recipe (replaces Section 10.1's loop mechanics)....11
7.3 Stage A-prime: realignment pre-training, and the diagnostics that guard it....12
8. Flaw 6: Earning the Right to Beat XGBoost - Cross-Asset and Micro-Sequence State....13
8.1 Confirmation: on flat features, trees win....13
8.2 Fix part one: the cross-asset panel (all free, all dimensionless)....13
8.3 Fix part two: ordered micro-sequence tokens....14

8.4 The honest control: race the transformer, gate on disagreement....14
9. Flaw 7: Walk-Forward Inference Compute Architecture....15
9.1 Corrected arithmetic: the GPU dissolves the problem, and exposes the real one....15
9.2 The fix: the two-kernel probability-table architecture....15
9.3 The real bottleneck: refits, not inference....15
10. Consolidated Specification Delta....16
11. Updated Phase 0 and Phase 1 Gate Checklist....17
12. Residual Risks and Open Questions....17
13. References....18
13.1 Labeling, calibration, and distributional methods....18
13.2 Looped and recursive transformer architecture....20
13.3 Market evidence, data, and platforms....21

1. Executive Summary and Verdict Table

This report is the engineering-verification pass that the owner ordered before allowing Phase 0/1 to start: a targeted deep-research and design review of the seven technical and quantitative flaws identified in the architectural audit of the AURUM build plan (the AURUM Decider Research Report and Build Plan, version 1.0, 6 November 2026). The mandate was to confirm or refute each flaw on evidence, produce a mathematically sound architectural fix for each, and consolidate the results into an updated specification with numeric acceptance gates, so that the build can be locked and executed without discovering these defects mid-training.

The verdict is that all seven flaws are real. Three of them would have silently corrupted training or evaluation and were the most urgent: nominal price levels in the serialized state act as calendar-date fingerprints (Flaw 1); the counterfactual-pair augmentation of Section 6.5 asks for labels that cannot be recomputed and would actively teach the model to ignore the flipped features (Flaw 2); and hindsight-optimal labels for the stop and exit tasks inject single-path noise exactly where calibrated probabilities are needed (Flaw 4). Two flaws degrade decision quality rather than corrupt data: the six-task cascade is circular, partly redundant, and six times too expensive at inference (Flaw 3), and the looped core as specified - frozen layers 13-16 with rank-16 LoRA - is under-parameterized to realign the pretrained residual-stream geometry across loop iterations (Flaw 5). The remaining two are effectiveness risks: the state as designed is a flat indicator summary on which gradient boosting will match the transformer (Flaw 6), and naive walk-forward evaluation computes 180,000 sequential forward passes that would take weeks (Flaw 7).

The fixes converge on one architectural idea: AURUM stops being a six-question chat and becomes a distributional world model with a deterministic decision layer. A single forward pass over the encoded state drives a barrier-matrix head that scores every (direction, stop width, reward ratio) combination, plus excursion-quantile and volatility heads; direction, stop, take-profit, gating, and Kelly sizing are then computed in closed form from one coherent, calibrated posterior - which dissolves the circularity, the redundancy, and the six-fold inference cost simultaneously. The counterfactual pairs are replaced by matched historical cohorts and regime-conditional bootstrap soft labels. The loop core is fully unfrozen (63M parameters) with per-iteration sandwich normalization, gated anchor re-injection, and a new realignment pre-training stage. The state is rebuilt dimensionless, cross-asset, and sequence-structured. And all out-of-sample inference is pre-computed once as batched GPU probability tables, exposing the true bottleneck - walk-forward refits, not inference - which the corrected budget shows fits inside one Kaggle quota week.

Table 1: Verdict on the seven audited flaws and the architectural response

#	Audited flaw	Verdict	Fix adopted (report section)
1	Nominal price levels and dollar ATR in the state are date fingerprints	Confirmed	Scale-invariance contract on every token; three-layer automated leakage audit (Section 3)
2	Counterfactual pairs flip a text tag but cannot recompute the path label	Confirmed	Matched-cohort augmentation + bootstrap soft labels; counterfactuals demoted to an evaluation metric (Section 4)
3	T1/T2/T4/T5 cascade is circular, T2 redundant, 6 passes per bar	Confirmed	Barrier-matrix world model, joint multi-task heads, single pass, deterministic EV/Kelly decision layer (Section 5)
4	Hindsight-optimal stop and exit labels are single-path noise	Confirmed	Outcome-indicator labels per barrier cell + MFE/MAE quantile heads with pinball loss (Section 6)
5	Frozen core + rank-16 LoRA cannot realign layers 13-16 across loop iterations	Confirmed	Full core unfreeze (63M) + per-iteration sandwich RMSNorm + gated anchor + Stage A-prime realignment (Section 7)
6	Flat 15-indicator state lets XGBoost match the transformer	Confirmed	Cross-asset panel + ordered micro-sequence tokens + honest GBDT control and ensemble gate (Section 8)
7	180,000 sequential forward passes would take weeks on CPU	Confirmed	One-pass batched GPU probability tables; two-kernel architecture; corrected refit budget (Section 9)

Every fix preserves the two founding principles of the plan: the model judges and code executes, and the unlooped r=1 model remains a strict special case of its own looped checkpoint so that recurrent depth can be adopted or dropped on evidence. Section 10 consolidates the specification deltas; Section 11 lists the revised Phase 0 and Phase 1 gates with numeric thresholds; Section 12 records what the fixes do not solve. With these changes adopted, the recommendation of this pass is that Phase 0/1 can be triggered: the plan is now internally consistent, leakage-auditable, and compute-feasible on the free stack.

2. Mandate, Method, and Evidence Base

The owner's instruction was explicit: hold Phase 0/1, run a targeted deep-research and engineering-verification pass on the seven specific problems, and return concrete, mathematically sound architectural fixes before any pipeline code is written. The verification followed the deep-research skill shipped in the DeerFlow repository (github.com/bytedance/deer-flow, cloned during the first research phase), whose four-stage methodology was applied as in the two prior AURUM reports: broad exploration of each problem territory, targeted deep dives per dimension with multiple query phrasings, diversity and validation across facts, examples, expert opinion, and criticisms, and a final synthesis check before any content was written.

The evidence base combines three layers. First, the two prior research reports (Project AURUM Deep Research Report; AURUM Decider Research Report and Build Plan), which

examined more than 180 primary sources across looped-transformer science, the Jev ecosystem, gold market structure, and validation statistics; their findings are reused here with their original confidence tags. Second, this pass ran roughly 30 new targeted search queries and verified in full or in relevant part about 20 additional primary sources covering recursive-transformer adaptation recipes, residual-stream geometry, meta-labeling, excursion analysis, quantile regression, matching estimators, the stationary bootstrap, adversarial validation, and batched inference throughput. Third, every quantitative claim that is specific to AURUM - parameter counts, memory budgets, FLOPs, wall-clock estimates, label-distribution statistics - was computed in a single reproducible script (audit_numbers.py) whose printed outputs are transcribed into this report; the hindsight-bias illustration in Section 6 is a 20,000-path simulation from the same script.

Confidence tags follow the prior convention: HIGH marks a primary source read in full or a computation reproduced locally; MEDIUM marks solid multi-source agreement; LOW marks single unverified claims, which are not relied upon anywhere in this report. Two scope notes. First, this pass changes the architecture specification but not the project's strategic posture: the honest target remains a net-of-cost Sharpe of 0.5 to 1.0, and the Deflated Sharpe Ratio and probability-of-backtest-overfitting gates remain the release authority. Second, the Jev reference video that the owner mentioned has still not arrived in the conversation; nothing in this report depends on it, and the design continues to follow the published Jev failure-mode documentation studied in the first research phase.

3. Flaw 1: Temporal and Price-Level Leakage in State Serialization

3.1 Confirmation: a price level is a date stamp

The audited fragment of Section 7.1 serializes the state with nominal numbers: 'last swing HH at 2612', 'ATR 6.2 (0.24%)'. Over AURUM's data window this is not a stylistic choice but a leakage channel. Gold traded near $300 in the early 2000s, near $1,800-2,000 from 2011 to 2023, broke out through 2024, and printed a January 2026 all-time high of $5,586 before the current correction to roughly $4,050. Price level is therefore a near-deterministic function of calendar date: a token sequence reading 2-6-1-2 in the digit-by-digit Qwen tokenizer identifies the 2024 regime about as reliably as a timestamp. Because the walk-forward design trains on earlier years and tests on later ones, a model that keys on level tokens can score well in validation by memorizing what the 2024 tape did after states that looked like 2,612 - and transfer nothing to a 2027 price region it has never seen. The dollar ATR has the same defect with an extra twist of incoherence: 'ATR 6.2' is 0.24 percent of price at $2,583 but only 0.12 percent at $5,166 - identical tokens describe different volatility physics depending on the unmarked price scale. The first research report already mandated

returns-based and binned features for exactly this reason; the Section 7.1 fragment violates that mandate, and the audit is correct to flag it as a critical flaw.

The formal statement of the fix is an invariance contract. Let P = (p_1, ..., p_t) be the entire raw price history available at decision time. Every numeric feature f in the serialized state must satisfy f(aP) = f(P) for any positive constant a - the state must be invariant under a rescaling of the price axis. Features that satisfy the contract are dimensionless functions of relative price movement: log returns, ratios of price differences to volatility, bounded oscillators, percentages, z-scores over trailing windows. Features that violate it - raw levels, raw differences, dollar volatility, dollar stops - are date fingerprints and are banned from the state text. The contract also covers time: hour-of-day and day-of-week are periodic and permitted; year, month index, and bar counters are banned.

3.2 Feature-by-feature audit of the Section 7.1 fragment

Table 2: Scale-invariance audit of every field family in the current Section 7.1 serialization

Field (current)	Invariant?	Replacement in the state text
'last swing HH at 2612'	No	Distance to last swing high: +1.6 ATR; retracement depth of the last leg: 62 percent
'ATR 6.2 (0.24%)'	Half	ATR percent only: 'ATR% 0.24'; the dollar figure is dropped everywhere
'price 1.8 ATR above EMA50'	Yes	Unchanged - this is the correct pattern for all distance fields
'RSI 63', 'ADX 31', Bollinger %B	Yes	Unchanged (bounded oscillators are already dimensionless)
MACD histogram (raw)	No	MACD histogram normalized by ATR or by price, then rounded
Returns over 1/4/12 bars	Yes (if log returns)	Keep; express in ATR units for cross-regime comparability (return / ATR%)
Session tag, day-of-week, hour	Yes	Keep; encode hour as two cyclical tokens (sin, cos)
Position: 'entry 2612'	No	Unrealized R (dimensionless), bars held, stop distance in ATR - never the entry price
Cross-asset levels (DXY, US10Y, VIX values)	No	Changes and z-scores only: dUS10Y in basis points, VIX z-score, DXY 24-bar return
Spread, commission, swap in dollars	No	Express in ATR units or basis points of notional (Section 5 uses 0.02-0.04 ATR round-turn)

The corrected serialization fragment below shows the same market moment as the audited example, fully dimensionless. Rounding is to fixed precision so token patterns repeat, and the format stays fixed and versioned, as the original plan required. The corrected fragment reads:

```
D1: trend=up (ADX 31), vol quartile 3/4, close +1.8 ATR above EMA50, retr
62%;
```

```
H4: RSI 63, MACD-hist +0.31 ATR (5 bars rising), dist to last swing high
+1.6 ATR;
H1: ATR% 0.24, ret24 +1.4 ATR, EMA20>EMA50, dist to EMA20 = 0.9 ATR;
M15: 3 up bars, session=LDN-NY overlap, dow=Wed, hour_sin 0.87, hour_cos
-0.50;
XAG: ret24 +0.9 ATR, XAU/XAG ratio z +1.8; EUR ret24 -0.2 ATR; JPY ret24
-0.3 ATR;
dUS10Y +7bp z+1.2, dReal10Y +5bp, VIX z +0.8, SPX ret24 -0.4 ATR;
position: flat; costs: 0.03 ATR round-turn, swap 0.10 ATR per 3 nights;
question: score all barrier cells over the next 48 H1 bars.
```

3.3 Fix part one: the stationarity contract as an engineering rule

The contract is enforced as eight rules written into the feature pipeline (SC-1 to SC-8). SC-1: every numeric token must be dimensionless; the serializer carries a whitelist of field types, and any field outside the whitelist fails the build. SC-2: all price differences are expressed in ATR units of the decision bar; all returns as log returns, optionally ATR-normalized. SC-3: volatility is expressed only as a percentage of price (ATR%) or as a quantile of its own trailing distribution. SC-4: macro and cross-asset series enter only as changes, z-scores, or quantiles - never levels - because yield and index levels also date-stamp the observation (the 10-year yield near 0.5 percent identifies 2020 as surely as $1,900 gold does). SC-5: time enters only through cyclical hour encoding, day-of-week, and session flags. SC-6: rolling statistics use strictly trailing windows (no centering, minimum-periods enforced), so no future bar contributes to a normalizer. SC-7: every fitted statistic that spans bars - z-score windows, quantile breakpoints, k-means codebooks for the Section 8 sequence tokens - is fitted on training-fold data only and frozen for the test fold. SC-8: the serializer emits a machine-readable schema hash; any change to the state format is a versioned event, because inconsistent phrasing is itself a distribution shift (the Laya lesson from the first research report).

Where memory matters, the float-side alternative is fixed-window fractional differentiation (Lopez de Prado, Advances in Financial Machine Learning, chapter 5): differencing the log-price series by a fractional order d below 1 preserves long memory while passing stationarity tests, and is the correct treatment for any feature consumed by the gradient-boosting control model rather than by the language model. For the serialized state itself, multi-horizon log returns over 1, 4, 12, 24, and 120 bars already carry trend memory in dimensionless form, so fractional differentiation is adopted for the tabular control features, not the text.

3.4 Fix part two: the three-layer automated leakage audit

A contract without a test is a hope. Three automated audits, each a gate, enforce it. Layer one is a scale-invariance unit test run on every data build: for a random sample of decision points, the entire raw input panel is multiplied by a in {0.25, 0.5, 2, 4}, features are recomputed from the rescaled panel, and the serialized state must be byte-identical to the original. Any diff

fails the build. This test is deterministic and catches accidental nominal fields that slip past the whitelist.

Layer two is adversarial validation, the standard technique for detecting train/test distribution shift: a gradient-boosted classifier is trained to distinguish training-period states from test-period states using only the serialized features. If the state is properly dimensionless, period is not decodable from the features, and the classifier's cross-validated AUC sits near 0.5; a 2023-versus-2026 AUC of 0.75 would mean the state still encodes the date and the model can still memorize the regime. Gate: adversarial-validation AUC at or below 0.55 on every train/test boundary, and a companion year-classification probe (predict the year bucket from the state) at or below 0.60 balanced accuracy. Layer three is a representation probe after training: a linear probe on the trained model's pooled final hidden states predicts the period bucket, with the acceptance threshold set relative to the probe trained on the deliberately leaky nominal-state variant - the leaky probe defines the ceiling, and the production model must sit far below it. The first report's contamination controls (from-scratch identical-architecture control, pre-2020-only training control, zero-shot knowledge probe of the base model) remain in force unchanged and are now joined by these three per-build audits.

> *Acceptance gates for Flaw 1: SC unit test passes on 100 percent of sampled decision points; adversarial-validation AUC at or below 0.55 at every walk-forward boundary; hidden-state period probe materially below the leaky-variant ceiling; schema hash versioned and frozen before Stage B training begins.*

4. Flaw 2: The Counterfactual Pair Paradox

4.1 Confirmation: the label cannot follow the flip

Section 6.5 proposes feature-perturbed counterfactual pairs: flip one tag of a state - D1 trend from up to down, session from London to Asia, volatility quartile shifted - and 'recompute the triple-barrier label on the perturbed path'. The audit is correct that this is impossible as stated. A triple-barrier label is a functional of the realized future path: $y = L(x, Z)$, where x is the state and Z is the M1/H1 path that actually followed. Flipping a text tag changes x but not Z, because the historical path is what it is. The resulting example (x', Z) with the old label $y = L(x, Z)$ is drawn from neither the factual distribution $p(x, Z)$ nor the interventional distribution of a genuine regime change - it is an off-manifold hybrid. Worse, the flipped state is internally contradictory: the D1 tag says downtrend while the H4 momentum fields, the EMA distances, and the 24-bar return of the very same state still describe the uptrend. Training on such pairs does not teach counterfactual reasoning; the gradient teaches the model to ignore the flipped feature, because the flipped feature is the one whose value does

not predict the (unchanged) label. The augmentation would actively destroy the influence of exactly the regime features the design cares about most.

The only way to obtain a valid counterfactual label is to observe (or credibly simulate) a future path that actually followed a state like x'. History contains millions of such states - just not at the same timestamp. That observation motivates the replacement.

4.2 Fix: matched cohorts, bootstrap soft labels, and a demoted counterfactual metric

The augmentation is rebuilt from three mathematically valid mechanisms. Mechanism one is matched-cohort augmentation, borrowed directly from covariate matching in the causal-inference literature (the Rubin potential-outcomes framework; Morgan and Harding 2006; Stuart 2010; Abadie and Imbens 2006, 2011). For an anchor state x with tag value v and realized outcome y, construct the counterfactual partner by retrieval rather than fabrication: search the training corpus for real historical states x_j whose target tag takes the flipped value v' and whose remaining covariates match the anchor - nearest-neighbor matching on the standardized feature vector with Mahalanobis distance, plus hard calipers on the fields that must not drift (volatility quartile, session, event proximity). The partner's label is its own realized outcome y_j - real physics, no fabrication. The training signal is a contrastive regularizer rather than a hard pair label: the model's direction distribution at x and at its matched partner x_j should differ in the direction the tag flip implies, and the loss penalizes failure to respect the flipped feature. Two diagnostics protect validity: an overlap (positivity) audit verifies that matched partners exist within caliper for at least 70 percent of anchors - in stressed regimes the flip may simply not co-occur with similar covariates, and those anchors are dropped rather than force-matched - and the median standardized match distance is reported with every dataset build. This is the honest version of what Section 6.5 wanted.

Mechanism two converts every label from a single-path draw to a path-distribution draw. For each decision point, the labeler draws B = 200 synthetic future paths with the regime-conditional stationary block bootstrap (Politis and Romano 1994): blocks of realized H1 returns are resampled from a pool restricted to the anchor's volatility quartile, session, and trend tag, with block length 24-48 bars so that short-lag dependence (volatility clustering, intraday seasonality) survives resampling. The triple-barrier outcome of every barrier cell is then evaluated on each bootstrap path, producing a soft label distribution q(win), q(loss), q(timeout) per cell instead of one hard draw. This simultaneously addresses Flaw 4: single-path label noise becomes an explicit distribution the model is trained to predict, and cross-entropy on soft targets is the correct loss for it. The honest caveats are recorded: block bootstrap preserves short-range dependence but not long-horizon regime persistence beyond the block; conditional pools reduce but do not eliminate this; and therefore a validation protocol is registered - a model trained on bootstrap-soft labels must not lose out-of-sample

log-loss against the same model trained on hard labels, or the soft-label machinery is switched off and only hard labels plus adjacent-cell smoothing remain.

Mechanism three demotes counterfactuals to what they can legitimately do: an evaluation metric, never a training signal. On out-of-sample data, for states where a matched partner exists, the system measures flip-consistency - when the D1 trend flips between two matched states, does the predicted direction flip or the confidence drop? The target is directional consistency above 65 percent on matched pairs, reported per tag type. The two augmentations that were already valid survive unchanged: paraphrase templates (identical numbers, different phrasing) attack rewording fragility, and 5-10 percent feature dropout (a field replaced by a neutral token) builds robustness to missing data without making any claim about labels.

4.3 Dataset budget update

The 120,000-example corpus is re-mixed as follows: approximately 82 percent factual examples with barrier-cell outcome labels (Section 5) and bootstrap-soft variants; 8 percent matched-cohort contrastive pairs (replacing the fabricated 10 percent counterfactual share); and 10 percent paraphrase and feature-dropout variants. The bootstrap labeler runs once, offline, on CPU kernels, and its outputs are versioned with the dataset. Fabricated-label counterfactual pairs are removed from the plan entirely.

> *Acceptance gates for Flaw 2: at least 70 percent of anchors matched within caliper (positivity audit); median standardized match distance below the registered threshold; flip-consistency above 65 percent on out-of-sample matched pairs; no out-of-sample log-loss degradation from soft labels versus hard labels in the registered A/B.*

5. Flaw 3: Circular Dependency in the Six-Task Cascade

5.1 Confirmation: three defects in one design

The audit identifies three distinct defects in the T1-T6 cascade, and all three are confirmed. First, the circularity. A triple-barrier label is defined by its barrier widths: whether a decision point is a win or a loss depends on the stop m and the profit target that the labeler fixed in advance. T1 (direction) is therefore trained on labels generated under one barrier configuration, while T4 (stop distance) later selects a different width at deployment - and T5 then feeds the Kelly formula a win probability p and payoff ratio b that describe a trade that is not the one being placed. The inputs to position sizing are exactly the one place where a mismatched probability is most dangerous, because sizing multiplies it into leverage. The original plan's own risk chapter calls miscalibrated probabilities 'miscalibrated leverage'; the cascade manufactures that miscalibration structurally.

Second, the redundancy. T2 (meta-gate, a learned noul over 'take this signal / pass') re-learns a threshold function of the same features that T1 already sees, and its probability is not independent of T1's softmax. Meta-labeling, as defined by Lopez de Prado (Advances in Financial Machine Learning, chapter 3), exists to attach a calibrated secondary probability to a primary signal that is rule-based and over-confident; it is a repair for non-probabilistic primaries. When the primary is itself a trained probabilistic model, stacking a second learned gate on top produces two heads trained on nearly the same target, conflicting gradients, and an opaque implicit threshold that cannot be tuned for risk appetite without retraining. Third, the cost. Six tasks asked as six prompts - and the Jev-style discipline of averaging over randomized option orders multiplies each question further - means six (or more) forward passes per decision bar, which at walk-forward scale is the compute problem the audit quantifies in Flaw 7.

5.2 The fix: a barrier-matrix world model with a deterministic decision layer

The fix answers the audit's own proposal affirmatively: yes, replace the cascade with a multi-task joint decision head, and yes, condition direction on a standardized barrier matrix. The redesigned model predicts, in a single forward pass, the outcome distribution of every standardized trade configuration - and nothing else. All six tasks become either a head over this distribution or a closed-form function of it. Concretely, define the barrier matrix as the Cartesian product of stop widths m in {1.0, 1.5, 2.0, 2.5} ATR and reward ratios R in {1.5, 2.5} (profit target = R x m), with a time barrier H of 48 H1 bars as the default vertical barrier. That is 8 cells per direction. For every cell the labeler records which barrier was touched first on the realized path - win, loss, or timeout - and the head learns P(outcome | direction, cell, state): 8 cells x 3 outcomes x 2 directions = 48 logits. Because the cells are fixed and standardized, the labels are well-defined before any model runs, and the circularity is gone by construction: the stop distance T4 used to predict is now a coordinate of the label space, not a downstream correction.

The head assembly uses hard parameter sharing in the multi-task-learning sense (Ruder 2017; Abdelsamie et al. 2026): one shared looped trunk encodes the state, and light heads read it. Head H1: the 48 barrier-outcome logits above. Head H2: timeout mark-to-market quantiles (q25, q50, q75 per direction), because a timeout is not a scratch - the position is closed at whatever the market gives, and the EV computation needs its expectation. Head H3: MFE/MAE excursion quantiles (Section 6) at horizons 24 and 120 bars. Head H4: realized-volatility forecast quantiles for the next 24 bars, which feeds the volatility-targeting overlay. Head H5: the procedural regime heads that T6 used to carry (volatility quartile, trend tag), kept as auxiliary tasks because cheap procedural labels regularize the trunk. Head H6 is not learned at all: the out-of-distribution score is computed from the loop architecture itself - the

divergence between successive loop iterations, the free uncertainty signal the looped design already provides - plus disagreement with the gradient-boosting control model of Section 8. Total continuous outputs: about 85, a trivial head budget.

Everything that used to be a separate task becomes a deterministic function of one coherent, calibrated posterior. The decision layer - code, not model - computes, for every direction and cell, the expected value in ATR units: EV(d, m, R) = p_win x (R x m) - p_loss x m + p_timeout x q50_MTM - cost, where cost is the round-turn expense of 0.02-0.04 ATR (Section 3's dimensionless cost encoding; $0.35 per ounce against an H1 ATR of $10-20) plus the expected swap for the horizon (0.05-0.10 ATR per three nights held). The trade taken is the argmax over all 16 direction-cell combinations; the gate is the requirement that the best EV clear a fixed positive threshold and that the OOD score sit below its ceiling and that the bar not fall inside an event blackout window. Sizing is fractional Kelly on the chosen cell's own numbers: with win probability p and payoff ratio b = R, the Kelly fraction is f* = p - (1 - p) / b, scaled by a quarter-Kelly multiplier k = 0.25 and hard-capped at 1.0 percent of equity per trade (computed examples: p = 0.55, R = 1.5 gives f* = 0.250 and quarter-Kelly 6.25 percent, so the cap binds; p = 0.50, R = 2.5 gives f* = 0.300). Because p, b, and the EV all come from the same predicted cell, the probability that sizes the trade is the probability of the trade - the consistency the cascade could not guarantee. This is decision-theoretically the correct division of labor: the model is a calibrated world model, and expected-utility maximization over its outputs is an auditable, tunable computation (the same 'model judges, code executes' principle the plan already committed to, now carried through the entire decision surface).

Table 3: Mapping of the original six tasks onto the barrier-matrix world model

Original task	Becomes	Where it lives
T1 Direction (choice)	argmax over direction of the best-cell EV	Decision layer, from head H1
T2 Meta-gate (noul)	EV threshold + OOD score + event blackout	Decision layer; loop-divergence and GBDT-disagreement signals
T3 Exit management (choice)	MFE/MAE quantile-derived rules; optional second-pass management head	Head H3 + code trailing rules (Section 5.3)
T4 Stop distance (choice)	the m coordinate of the argmax cell	Decision layer, from head H1
T5 Position size (score)	fractional Kelly f* = p - (1-p)/R on the chosen cell, quarter-Kelly, capped	Decision layer
T6 Regime check (noul)	procedural auxiliary heads (vol quartile, trend tag)	Head H5, training-time regularizer and live sanity display

5.3 Exits: derived rules first, a management pass second

Exit management deserves its own treatment because it is the one task whose input genuinely depends on the position (entry, unrealized R, bars held) and therefore cannot be fully pre-computed from flat-state passes. The MVP design keeps exits deterministic and position-agnostic: a trailing stop parked at the predicted q25 of MAE (giving the trade the room the state says it typically needs), partial profit-taking at the predicted q50 of MFE, a time stop at the barrier horizon, and the hard stop at the chosen cell's m. All of these are closed-form functions of heads H1-H4 and the position record, so they run in the same CPU decision layer as everything else. As a registered ablation, a learned management head can be added as a second inference pass over only the bars where a position is actually open - about 30 percent of bars in swing cadence - with the position described in dimensionless terms (side, unrealized R in quantile buckets, bars-held bucket, remaining stop distance in ATR). That two-phase design is what Section 9's compute plan assumes: one full entry pass over all bars, plus one management pass over the in-position subset.

5.4 Calibration, the property that makes EV arithmetic meaningful

Expected-value arithmetic is only as good as the calibration of the probabilities feeding it, so calibration is instrumented per head: temperature scaling fitted on a purged calibration split per checkpoint (Guo et al. 2017 - a single parameter, surprisingly effective), reliability diagrams per barrier cell family, and - the metric that actually guards the Kelly input - decision-conditional calibration: among trades the decision layer actually selects, binned by predicted win probability, the realized win rate must match the prediction. The registered gates: expected calibration error below 0.05 per major head on purged out-of-sample folds, and decision-conditional reliability slope within 0.8-1.2. The prior plan's monthly temperature refits and per-regime reliability monitoring carry over unchanged.

> *Acceptance gates for Flaw 3: per-head ECE below 0.05 on purged OOS; decision-conditional reliability slope 0.8-1.2; single forward pass replaces all six task queries with at most 2x the r=1 cost; EV-argmax cell changes monotonically with injected cost changes (a sanity property the cascade could not offer).*

6. Flaw 4: Single-Path Hindsight Noise in Exit and Stop Labels

6.1 Confirmation: hindsight-argmax is a biased, noisy estimator

The audited labels for T4 ('the widest stop not hit before the profit barrier') and T3 ('the hindsight-optimal action over the next K bars') are argmax functionals of a single realized path. The mathematical defect is elementary and fatal: for any utility u and path noise Z, E[argmax over m of u(Z, m)] is not equal to argmax over m of E[u(Z, m)] - the expectation of a maximum is inflated by the noise it is taken over, the winner's-curse effect, and the selected argmax concentrates on the luck of the draw rather than the state's systematic properties. In

plain terms: two adjacent bars with statistically identical states receive wildly different 'optimal stop' labels because their individual wicks differed, so the label attached to a state is mostly noise. A model trained on such targets cannot fit them (they are unpredictable by construction), and the residual it does fit is a blurred average that destroys exactly the probability sharpness - the Brier score, the reliability curve - that the whole design depends on. The same argument applies to hindsight-optimal exit actions, where 'what I should have done' is computed with knowledge of the future that the state does not contain.

6.2 The fix: train on outcome distributions, decide on expectations

The principle that replaces both labels: train only on outcome variables that are unbiased samples of the state's conditional distribution, and take argmaxima at decision time over predicted expectations, never inside the labels. For stops, the fix is already embedded in Flaw 3's barrier matrix: the per-cell outcome indicators (win/loss/timeout on the realized path) are unbiased draws, cross-entropy on them estimates P(outcome | state, cell) correctly, and the optimal stop is the decision layer's argmax over predicted EV - computed after prediction, from calibrated numbers, not baked into a noisy label. Adjacent-cell label smoothing (a triangular kernel that shares a fraction of mass between neighboring stop widths) reflects that the true optimum is not knife-edge and measurably improves calibration (Muller, Kornblith and Hinton 2019), and the bootstrap soft labels of Section 4 provide the path-distribution version of the same smoothing.

For exits, the hindsight-optimal action label is replaced by excursion distributions, the trade-analysis machinery practitioners already use for exactly this question (Maximum Favorable and Adverse Excursion, introduced by John Sweeney and standard in trade analytics platforms). For each decision point and horizon h in {24, 120} bars, the labeler records MFE(h) and MAE(h) - the largest favorable and adverse excursion in ATR units - and the model predicts their conditional quantiles (q25, q50, q75) with the pinball loss of quantile regression (Koenker and Bassett 1978), which targets quantiles directly and coherently across levels. Exit policy then reads the quantiles: trail at q25 of MAE, scale out at q50 of MFE, time-stop at the horizon, hard stop from the chosen barrier cell. This is the distributional-prediction posture of distributional reinforcement learning (Bellemare, Dabney and Munos 2017; Dabney et al. 2017), transplanted from return distributions to excursion distributions: model the distribution, act on its statistics. As a registered ablation, a per-action continuation-value variant (regression on each exit action's realized R-multiple, with the argmax taken at decision time) is retained in the experiment matrix; what is deleted permanently is the hindsight-argmax label itself.

6.3 The bias, measured

The audit's claim is not merely theoretical; the companion computation makes it concrete. The audit_numbers.py simulation draws 20,000 price excursion paths from one identical market state (per-bar drift +0.012 ATR, per-bar standard deviation 0.35 ATR, 48-bar horizon, reward ratio 1.5) and evaluates the triple-barrier outcome at every stop width. The population answer is unambiguous: expected P&L per trade rises from +0.219R at a 1.0x ATR stop to +0.336R, +0.442R, and +0.503R at 1.5x, 2.0x, and 2.5x - the EV-optimal stop is 2.5x ATR for this state. The hindsight-argmax label, computed per path, lands on 1.0x for 40.8 percent of paths, 1.5x for 19.3 percent, 2.0x for 17.6 percent, and on the EV-optimal 2.5x for only 22.3 percent of paths - barely different from the 25 percent a coin would give. A supervised model given these labels sees near-maximal label entropy conditioned on a constant state: there is nothing to learn but noise. The same script drives Figure 1.

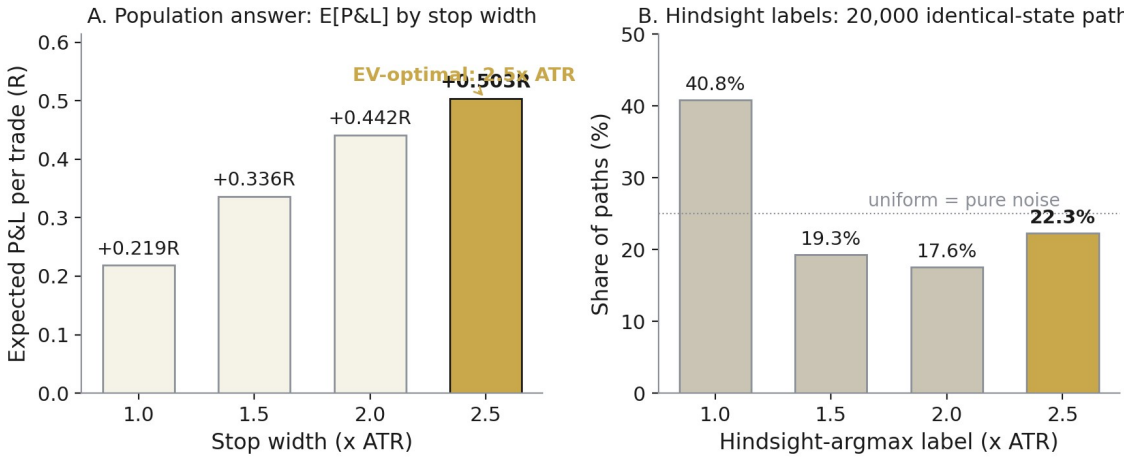


Figure 1: The hindsight-argmax label problem, measured on 20,000 simulated paths from one identical state. Panel A - the population answer: expected P&L per trade by stop width; the EV optimum is 2.5x ATR (+0.503R). Panel B - the label a hindsight-labeling scheme would assign: the argmax per path scatters across all four widths (40.8 / 19.3 / 17.6 / 22.3 percent), landing on the EV-optimal width for only 22.3 percent of paths - statistically indistinguishable from noise at a constant state. Simulation: drift +0.012 ATR/bar, sigma 0.35 ATR/bar, 48-bar horizon, R = 1.5.

Calibration instrumentation completes the fix. Per-head reliability diagrams and Brier scores are computed on purged out-of-sample folds for the barrier head (aggregated and per cell), the excursion quantile heads (coverage tests: does the realized MAE fall below the predicted q25 about a quarter of the time), and the timeout-MTM quantiles. Decision-conditional calibration (Section 5.4) is the binding gate, because it is computed on the distribution the decision layer actually samples. The registered ablation comparing outcome-distribution training against hindsight-argmax training on identical splits will quantify the Brier and ECE gap directly; the simulation above predicts the direction of that gap.

> *Acceptance gates for Flaw 4: per-cell ECE below 0.05 on purged OOS; quantile coverage within 3 percentage points of nominal for every excursion quantile head; the registered hindsight-label ablation shows outcome-distribution training strictly better on Brier and decision-conditional reliability.*

7. Flaw 5: Looped Pretrained Latent Space Mismatch, Layers 13-16

7.1 Confirmation: the residual stream is depth-coded, and looping violates the contract

The audit's mechanism is confirmed by the interpretability literature. In a pretrained decoder-only transformer, the residual stream is a running sum of layer outputs, and its norm grows systematically - approximately exponentially - with depth: measurements on GPT-2 XL show an average per-layer growth factor of about 1.045, and 2026 analyses of residual-stream geometry and norm-agnostic depth scaling make the same point for modern architectures. Layer 13 of Qwen3-0.6B therefore expects inputs with the statistics of a layer-12 output, and layer 16 emits outputs whose statistics are tuned for consumption by layer 17 - larger norm, later subspaces. The AURUM loop feeds s_1, a layer-16-space output, back into the layer-13 input (through the 2048-to-1024 adapter), so iterations r = 2 and r = 3 run the borrowed layers far off their pretrained operating distribution. The identity-dominant adapter initialization is a sound ReZero-style stability trick (iteration 1 reproduces the unlooped trunk exactly), but it only delays the confrontation: as training opens the feedback path, a frozen core with rank-16 LoRA - roughly 4 million adapted parameters against 63 million of frozen function - lacks the capacity to re-map the input-output contract across iterations. The best a frozen core can do with feedback it cannot process is attenuate it back toward identity, spending the extra loop compute on nothing.

The external evidence is unambiguous: every published system that loops or reuses pretrained layers successfully paid real adaptation compute. LoopUS converted Qwen3-1.7B with about 3 billion training tokens; the Retrofitting-Recurrence surgery used 25-52 billion; Relaxed Recursive Transformers (Bae et al., ICLR 2025) needed extended continued training at 60 billion tokens plus knowledge distillation for its recursive Gemma 1B to recover 58.4 percent average accuracy against the full model's 58.6 - and critically, it found that strict weight tying underperforms per-depth adaptation: giving each loop iteration its own LoRA module (multi-LoRA), with SVD-based initialization from the layer residuals worth up to 6.5 points over zero-init, is what made recursion competitive (recursive Gemma 1B beats a vanilla Gemma 1B by up to 13.5 points). Huginn trained its core from scratch precisely so the core's input-output contract is self-consistent under iteration - 800 billion tokens of budget we do not have. And the zero-training option (Inner Loop Inference, looping a frozen model with representation interpolation) yields only modest gains - a fallback, not a recipe. The audit's question - is rank-16 LoRA on frozen layers enough? - has a documented answer: no. Full unfreeze of the 63M-parameter core is both affordable and necessary; the user's own VRAM estimate is confirmed below with exact numbers.

7.2 The realignment recipe (replaces Section 10.1's loop mechanics)

Five coordinated changes convert the loop from borrowed-weights-in-a-cycle into an adapted recurrent operator. Change one: unfreeze the recurrent core fully. Layers 13-16 of Qwen3-0.6B contain 62.9 million parameters (four layers at 15.73 million each: attention projections of 6.29 million plus SwiGLU MLP of 9.44 million per layer, computed for hidden size 1024, GQA 16Q/8KV at head dimension 128, intermediate 3072). The prelude (layers 1-12) and coda (17-20) stay frozen, optionally with rank-16 LoRA on the coda. Change two: per-iteration sandwich RMSNorm. Every loop iteration r gets its own pre-norm at loop entry and post-norm at loop exit - 8,192 parameters in total - so each iteration's input statistics are re-standardized before the core and its output re-standardized before the next consumer, decoupling the depth-coded statistics of the borrowed layers from the loop's dataflow. RMSNorm is the model's native normalization, so the insertion is architecturally native. Change three: gated anchor re-injection replaces the 2048-to-1024 concatenation adapter. The loop input at iteration r is h_r = N_pre((1 - lambda_r) x e + lambda_r x W_s x s_(r-1)), where e is the prelude output (the anchor), W_s is a 1024x1024 feedback projection initialized near-orthogonal, and lambda_r is a learned scalar initialized at zero. At initialization the model is exactly the r=1 baseline (ReZero logic, Bachlechner et al. 2020); training opens lambda_r smoothly, and the anchor e is re-injected at every iteration - Huginn's input re-injection pattern - so the state cannot drift away from the question being refined. Change four: per-iteration LoRA deltas, rank 8, on the core's attention and MLP projections for iterations r = 2, 3, 4 - about 2.2 million parameters total - mirroring Relaxed Recursive's finding that each reuse site needs its own low-rank correction, while the unfrozen shared weights carry the general refinement operator. Change five: the core remains blind to its iteration index in the weights (the per-iteration signal lives in the norms, the LoRA deltas, and lambda_r), which preserves the property that inference can run any r with the same checkpoint.

```
e      = prelude(x)                    # layers 1-12, frozen
s_0    = 0 ;  lambda_r = 0 (learned, ReZero-style)
for r in 1..R:                         # R sampled from {1,2,3} in training
  h_r    = N_pre_r( (1-lambda_r)*e + lambda_r * W_s * s_(r-1) )
  u_r    = Core( h_r )                 # layers 13-16, UNFROZEN + per-r
LoRA r=8
  s_r    = N_post_r( u_r )
z      = coda( N_out(s_R) )            # layers 17-20, frozen (+ optional
LoRA)
heads  = {barrier48, MTMq6, MFE/MAE24, vol3, regime4}   # ~85 outputs
loss   = CE(barrier cells, soft labels) + pinball(quantiles)
        + aux(regime) + KL_anchor(z_r=1 logits || original Qwen3 logits)
```

7.3 Stage A-prime: realignment pre-training, and the diagnostics that guard it

Unfreezing alone is not enough; the loop must be re-taught to be a loop. A new training stage, Stage A-prime, inserts realignment continued pre-training between the head-sanity stage and decision SFT: 150-300 million tokens of next-token prediction on a mixed corpus (roughly 50 percent serialized market states and market-adjacent text, 30 percent synthetic numeric-reasoning sequences, 20 percent generic replay to prevent drift), with a KL-anchor term tying the r=1 logits to the frozen original model on the same inputs - a distillation leash that prevents the unfrozen core from wandering while it learns to accept feedback. Supervision follows the LoopUS random-depth recipe: each training example backpropagates from a randomly sampled r in {1, 2, 3} with geometric weights, which the loop-retention literature shows preserves quality across inference depths (99.8 percent retention with random loop-position supervision in the 2026 study, versus 22 percent without). The budget is computed, not hoped for: at the conservative 1,500-4,000 tokens per second a hand-rolled looped forward-backward achieves on one T4, and the measured 1.8-2.4x speedup of two-T4 DDP, Stage A-prime at 150 million tokens costs 4.3-15.4 GPU-hours - one Kaggle week accommodates it together with Stage B.

Four diagnostics are logged per checkpoint and gated. Per-iteration cosine similarity between s_r and the anchor e should start near the r=1 value and decrease gradually - a collapse to 1 means the loop is not computing anything new; a collapse to 0 means it stopped reading the question. The lambda_r trajectories should open into a sane band (final values roughly 0.2-0.8); values pinned at 0 mean the loop never engaged, and values near 1 mean the anchor was abandoned. The fixed-point residual (norm of s_r minus s_(r-1)) should decay across iterations without exploding - the deep-equilibrium intuition (Bai et al. 2019) that a stable loop converges toward a fixed point. And per-r head metrics must show the r=3 model matching or beating r=1 on validation proxy tasks, which is the entire economic point of looping. The fallback ladder is unchanged in spirit: if the unfrozen loop will not train stably, freeze the core and keep only per-iteration LoRA rank 16 plus the sandwich norms (medium capacity); if that also fails, the zero-training Inner-Loop interpolation; and if that adds nothing, ship r=1 - the unlooped model remains the deployable special case of its own checkpoint family, as the plan has promised from the start.

Table 4: Parameter and memory budget of the realignment recipe (computed; one T4 = 15.4 GB usable)

Component	Parameters	Memory contribution
Frozen trunk, layers 1-12 and 17-20 (fp16)	251.7M	0.50 GB
Frozen tied embeddings (fp16, input side only)	155.6M	0.31 GB
Unfrozen core, layers 13-16	62.9M	0.15 GB fp16 copy
Per-iteration LoRA r=8 (r = 2,3,4)	2.16M	in trainable set
Sandwich norms + W_s + lambda gates + heads	~10.1M	in trainable set

Component	Parameters	Memory contribution
Total trainable	75.1M	1.20 GB (fp32 master + AdamW moments + fp16 grads)
Activations, batch 16 x 600 tokens, checkpointed, 28 layer-passes	-	~1.7 GB
Peak estimate	-	~3.7 GB of 15.4 GB - single T4 comfortable; 2x T4 DDP halves per-device load

> *Acceptance gates for Flaw 5: Stage A-prime held-out NLL within 2 percent of the r=1 baseline; r=3 at least matches r=1 on validation proxy metrics (barrier-head log-loss and ECE); lambda_r settled in 0.2-0.8; fixed-point residual decaying without explosion; throughput within 3x of the plain r=1 forward-backward on the Phase 0 smoke test.*

8. Flaw 6: Earning the Right to Beat XGBoost - Cross-Asset and Micro-Sequence State

8.1 Confirmation: on flat features, trees win

The audit's premise is the best-documented result in applied tabular machine learning: on medium-sized numeric tabular data, gradient-boosted trees remain state of the art (Grinsztajn, Oyallon and Varoquaux, NeurIPS 2022, arXiv 2207.08815 - 45 datasets, 20,000 GPU-hours of hyperparameter search). The first AURUM research report added the financial corroboration: the eighth-place Jane Street solution reported explicitly that MLPs, time-series transformers, cross-symbol attention and embeddings 'did not work for me', and time-series foundation models lose to classical methods on half of deployment datasets. If AURUM's state is a flat summary of roughly fifteen lagging indicators - RSI 63, ADX 31, distances, tags - the looped 450M-parameter model is being asked to process tabular data, the one regime where trees are favored, while paying a 450M-parameter compute tax. The audit is correct: either the state acquires structure that trees cannot see, or the transformer loses to its own control model.

8.2 Fix part one: the cross-asset panel (all free, all dimensionless)

Gold is not priced in a vacuum, and the drivers are precisely the ones the free data stack already covers. The academic anchor is Erb and Harvey's The Golden Dilemma (2013): the real price of gold has a long-run negative relationship with real interest rates - the single most-cited macro relationship in the gold literature - and gold's long-run real return is near zero with strong mean reversion, which is exactly the kind of slow-moving cross-asset context a state should carry. The dollar side is equally established: gold and the dollar index show a significant medium-to-long-term inverse relationship, regime-dependent and

nonlinear (2026 SSRN work and the nonlinear-dynamics literature agree on the sign with regime caveats). Silver gives a relative-value channel: the gold-silver ratio (about 68 at the October 2026 quotes of $4,170 gold and $61 silver) mean-reverts on multi-month horizons and is computable from the same M1 feeds as gold. The panel below lands inside the Section 3 contract - every series enters as changes, z-scores, or ATR-normalized returns, never levels - and every source is already in the project's verified free stack.

Table 5: The cross-asset panel added to the state (sources verified in the first research phase)

Series	Source (free)	Granularity	Encoded as (dimensionless)
XAGUSD (silver)	HistData / Dukascopy M1	H1	ret24 in ATR units; XAU/XAG ratio z-score (24 and 120 bars)
EURUSD, USDJPY	Dukascopy M1	H1	ret24 in ATR units; synthetic dollar factor; rolling correlation to XAU (24/120 bars)
DXY dollar index	FRED DTWEXBGS / yfinance DX-Y.NYB	Daily	24-bar return, z-score; forward-filled with 1-day lag
US 10Y nominal yield	FRED DGS10	Daily	Change in basis points, z-score
US 10Y real yield (proxy)	FRED DGS10 minus T10YIE	Daily	Change in basis points, z-score (Erb-Harvey channel)
VIX	FRED VIXCLS	Daily	Level z-score, 1-day change
S&P 500	FRED SP500	Daily	24-bar return in ATR units (risk appetite)
WTI crude	FRED DCOILWTICO	Daily	24-bar return in ATR units (commodity complex)

Daily macro series are joined to H1 bars with a strict one-day publication lag (the FRED value for day t enters states from day t+1 onward) and an explicit staleness tag, because macro data is the easiest place in the whole pipeline to hide look-ahead bias. The panel's purpose is not decoration: the divergence combinations it exposes - dollar up with gold up (real-bid regime) versus dollar down with gold up (weak-dollar rally), real yields spiking with silver breaking down - are genuinely different states with different follow-through, and they are interactions a tree can only see if a human engineers them, but which a shared trunk learns as aligned token groups.

8.3 Fix part two: ordered micro-sequence tokens

The second structural addition gives the transformer the one thing its architecture is built for and a flat feature vector is not: ordered sequences with local semantics. The design follows the patching principle validated by PatchTST (Nie et al., ICLR 2023, arXiv 2211.14730): subseries-level tokens retain local semantic information while cutting attention cost, and were the design that finally made transformers competitive on long-horizon forecasting. Concretely, the last 30 M1 bars before each H1 decision are serialized as an ordered token

run: each M1 bar maps to one shape code from a 48-entry codebook fitted by k-means on training-fold normalized bar shapes (body position, range, direction), plus a signed return-quantile token and a tick-volume z-token. The last 8 H1 bars are likewise serialized as ordered tokens (shape code, return quantile, volume quantile, session id) rather than collapsed into aggregates. The recent sequence then carries run-lengths, compression and expansion, breakout retest patterns and momentum acceleration - structure that attention can read positionally and that a gradient-boosted tree ingesting the same fifteen summary floats cannot see without a human pre-computing every interaction. Codebook fitting is training-fold-only (rule SC-7), and the codebook is versioned with the dataset.

Table 6: Revised 600-token state budget (computed; base case with headroom for paraphrase variants)

State block	Tokens	Content
XAU multi-timeframe core	110	M15/H1/H4/D1 indicators, all dimensionless (Table 2 replacements)
H1 bar history (ordered)	48	Last 8 bars x 6 tokens: shape code, ret quantile, vol quantile, session
M1 micro-sequence tail (ordered)	60	Last 30 M1 bars x 2 tokens: shape code + signed ret quantile
Cross-asset panel	36	Table 5 series, ~4 tokens each, grouped by asset with id tags
Macro block	12	Real yield, DXY, VIX, SPX, WTI deltas and z-scores
Session / time / event	14	Hour sin-cos, day-of-week, session flags, minutes to next FOMC/NFP/CPI
Position context (when in trade)	12	Side, unrealized R bucket, bars held, stop distance in ATR
Header and task framing	20	Fixed versioned template tokens (schema hash)
Base total	312	Grows to ~350-420 with paraphrase variants - inside the 600 budget

8.4 The honest control: race the transformer, gate on disagreement

The gradient-boosting control is promoted from baseline to permanent racing partner, for three reasons. First, as the ablation control it gets the identical flat features plus cheap derived sequence aggregates (run-length of directional bars, last-hour range percentile), so that if the looped model still wins out-of-sample, the win is attributable to the sequence structure and the trunk, not to information the control never saw. Second, as an ensemble member it is the cheap second opinion the verified individual winners all converged on - the Jane Street second place derived a third of its final score from a 17-model ensemble plus online updates, and the Optiver solo winner blended CatBoost, a GRU and a transformer. Third, as a gate: when the transformer and the GBDT control disagree on direction at the chosen cell, the state is one the two model families read differently, and the default action is stand-aside -

disagreement is uncertainty, and uncertainty at 0.55-class edges is a no-trade. The loop-depth divergence signal (Section 5) and the GBDT disagreement score together form the OOD gate's two inputs.

> *Acceptance gates for Flaw 6: looped model beats the identically-fed GBDT control on the decision metric (net EV capture and barrier-head Brier) on purged OOS; removing the sequence blocks degrades the looped model measurably (registered ablation proving the structure earns its tokens); cross-asset staleness audit shows zero same-day macro joins.*

9. Flaw 7: Walk-Forward Inference Compute Architecture

9.1 Corrected arithmetic: the GPU dissolves the problem, and exposes the real one

The audit's arithmetic is confirmed: 30,000 H1 bars x 6 task queries at loop depth r = 3 is 180,000 sequential forward passes, and at the 7-40 seconds per decision the first report measured for a 0.6B looped model on CPU, that is 15-83 days of wall clock - a timeout, not a backtest. The joint-head fix of Section 5 removes the factor of six at the source: one pass per bar for entry decisions plus a management pass on the roughly 30 percent of bars where a position is open. The remaining compute is then a batching problem, and batching is what GPUs do. The forward cost of the 20-layer trunk is about 0.63 GFLOP per token at r = 1, rising to 0.88 GFLOP per token at r = 3 (28 layer-passes); 30,000 states x 600 tokens is 18 million tokens, so a full r = 3 sweep is 15.9 PFLOP - 11 to 20 minutes on a single T4 at the conservative 13-23 TFLOPS effective throughput that batched fp16 SDPA inference achieves. Producing probability tables for all three loop depths r = 1, 2, 3, entry plus management, totals about 53 PFLOP, or 0.6 to 2.3 hours on one T4. Both T4s halve that. The complete out-of-sample inference layer of the project costs less GPU time than one percent of a weekly quota - provided it is engineered as the two-kernel design below rather than as per-bar calls.

9.2 The fix: the two-kernel probability-table architecture

Kernel A, the OOS table builder, runs on GPU and does nothing but turn features into probability tables. Its stages: load the versioned feature parquet; serialize all decision-point states once (orjson, schema-hashed); tokenize in batches with the fast tokenizer; run batched fp16 inference under torch inference mode with SDPA attention, batch size 128, pinned memory, two data-loader workers, and a resume checkpoint every 2,000 batches so a 12-hour session cap or a preemption never costs a rerun; collect the head outputs (48 barrier logits, quantile heads, vol, regime, the loop-divergence OOD score, and the r = 1, 2, 3 variants from the same batch) plus the GBDT control's probabilities computed on the same rows; and write one row per (timestamp, fold id, r) to a compressed parquet - the probability table - published

as a versioned private Kaggle Dataset. No generation, no per-sample Python overhead, no model.generate anywhere: this is single-forward classification, which is the cheapest thing a transformer does.

Kernel B, the backtest runner, is CPU-only and consumes the table without ever touching the model. The deterministic decision layer evaluates EV per direction and cell (costs, swap, blackout windows), applies the gate and the fractional-Kelly cap, and emits a trade list; vectorbt then runs the vectorized execution sweeps, and the NautilusTrader realism pass re-executes the final configuration on bid/ask data. The decisive property is that every policy variant - cost multipliers at 1.0x, 1.5x, 2.0x; loop depths r = 1, 2, 3; gate thresholds; Kelly fractions; each walk-forward fold's checkpoint - is a re-read of the same table with different post-processing parameters. Inference happens once; the entire experiment matrix of Phase 4 is then a CPU workload that costs zero GPU quota. This is the audit's requested batched GPU pre-computation, made concrete.

9.3 The real bottleneck: refits, not inference

Correcting the compute picture honestly exposes where the true constraint lives: walk-forward retraining. A single Stage B decision SFT over the 120,000-example corpus is 144 million tokens for two epochs - 4.2 to 14.8 GPU-hours on two T4s at the assumed 1,500-4,000 tokens per second per device and the measured 1.8-2.4x DDP speedup. Six cold refits for a full combinatorial purged CV would cost 26-92 GPU-hours before any ablation - not one quota week. The adopted schedule is therefore: one anchor model trained on the earliest regime blocks, three warm-start refits (each continuing from the previous checkpoint for half an epoch on newly arrived data, 1.0-3.7 GPU-hours each), probability tables per fold, and the registered ablations. The complete walk-forward campaign lands at 12-37 GPU-hours - one to one-and-a-half quota weeks, with the fold structure, embargo discipline, Deflated Sharpe Ratio and probability-of-backtest-overfitting gates of the original validation protocol unchanged; what changes is that fold count is honest about the free tier's budget, and every configuration trial is counted exactly as the multiple-testing chapter demands. Figure 2 shows the budget.

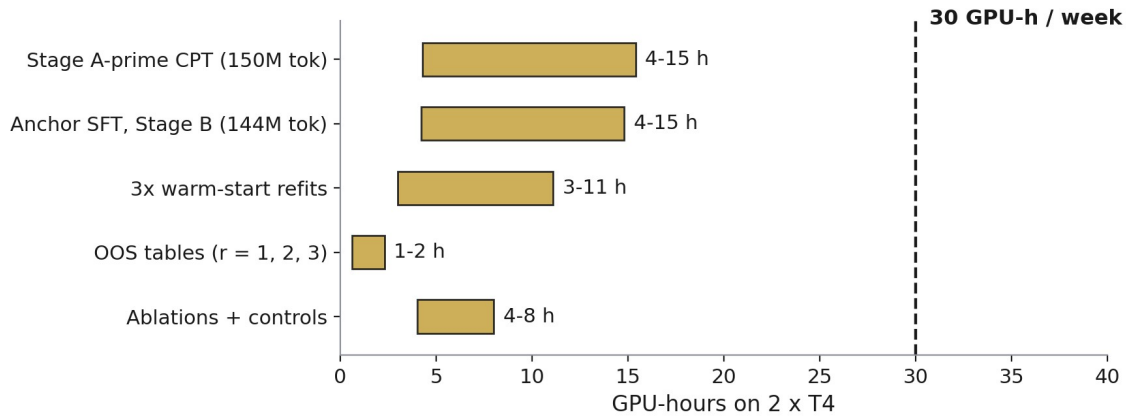


Figure 2: The corrected walk-forward compute budget on Kaggle's free tier (2 x T4). The full campaign - Stage A-prime realignment, anchor SFT, three warm-start refits, all OOS probability tables at r = 1, 2, 3, and the ablation suite - totals 12-37 GPU-hours against the 30-hour weekly quota. Full CPCV with six cold refits (26-92 GPU-hours) is the infeasible alternative and is replaced by warm-start refits under the same statistical gates.

Two acceleration options are registered as optional, not load-bearing. ONNX Runtime with fp16 has delivered roughly 2.9x throughput gains over eager PyTorch in production NLP serving, with the largest wins at small batch sizes; exporting the looped model with r unrolled is a Phase 4 optimization only if the table builder misses its two-hour gate. And torch.compile on the hand-rolled looped forward is worth one smoke-test measurement in Phase 0, with no commitment.

> *Acceptance gates for Flaw 7: Phase 0 smoke test measures actual looped inference tokens per second on T4 (gate: full r = 3 table for 30,000 bars in at most 2 hours on one T4); resume-from-checkpoint verified by killing the kernel mid-run; probability-table schema versioned and frozen before Phase 4.*

10. Consolidated Specification Delta

The table below is the authoritative diff against the AURUM Decider Research Report and Build Plan (version 1.0). Sections not listed are unchanged. The intent is that this table, together with the acceptance gates of Section 11, is what gets frozen as version 1.1 of the build plan before Phase 0 begins.

Table 7: Section-by-section specification delta (version 1.0 to version 1.1)

Plan section	Version 1.0	Version 1.1 (this pass)
6.4 Features and labels	Triple-barrier labels at fixed m, n; hindsight exit and stop labels	Barrier-matrix labels over 8 standardized cells x 3 outcomes x 2 directions; MFE/MAE and timeout-MTM quantile labels; FFD features for the tabular control
6.5 Augmentation	Counterfactual pairs with 'recomputed' labels (10 percent)	Removed; replaced by matched-cohort contrastive pairs (8 percent) + bootstrap soft labels (B = 200) + paraphrase and feature dropout

Plan section	Version 1.0	Version 1.1 (this pass)
7.1 State serialization	Nominal numbers allowed ('swing HH at 2612', 'ATR 6.2')	Scale-invariance contract SC-1..SC-8; corrected dimensionless fragment; three-layer leakage audit (invariance test, adversarial validation, representation probe)
7.2 Six-task decision map	T1-T6 as six prompted tasks in cascade	Barrier-matrix world model: joint heads (~85 outputs), one forward pass; tasks become heads or decision-layer functions (Table 3)
7.3 Probabilities to positions	Kelly on p and b from different tasks	EV-argmax over cells net of ATR-unit costs; fractional Kelly f* = p - (1-p)/R on the chosen cell; quarter-Kelly, 1 percent cap
10.1 Loop mechanics	Frozen trunk + rank-16 LoRA; concat adapter 2048-to-1024	Unfrozen 63M core; per-iteration sandwich RMSNorm; gated anchor re-injection with ReZero lambdas; per-iteration LoRA r=8
10.3 Parameter budget	~12-15M trainable	75.1M trainable; peak VRAM ~3.7 GB of 15.4 GB on one T4 (Table 4)
10.4 Training recipe	Stages A (head sanity), B (SFT), C (full unfreeze escalation)	Stage A unchanged; new Stage A-prime realignment CPT (150-300M tokens, KL anchor, random-r supervision); Stage B joint-head SFT; Stage C unchanged as escalation
10.5 Dataset budget	120k examples, 45 percent T1/T2, counterfactual 10 percent	120k examples re-mixed (Section 4.3); labels per barrier cell + quantiles; matched cohorts 8 percent; paraphrase/dropout 10 percent
State content (6.4/7.1)	Single-asset, ~15 indicator summaries	Cross-asset panel (Table 5) + ordered M1 and H1 sequence tokens (Table 6); base 312 tokens of 600
9.6 / Phase 4 backtest	Per-bar policy queries during backtest	Two-kernel architecture: GPU probability tables once, CPU decision layer + vectorbt sweeps offline; all variants from one table
Baselines (9.4)	XGBoost as one of five baselines	GBDT control promoted to racing partner + disagreement gate + r=1 ablation; ensemble reporting

Two parameters are deliberately left open for the owner, as in version 1.0: the vertical barrier H (default 48 H1 bars, to be revisited against the multi-horizon experiment matrix of the first research report) and the Kelly multiplier k (default quarter-Kelly, 0.25, with the 1.0 percent hard cap). Everything else in this table is proposed for immediate freeze.

11. Updated Phase 0 and Phase 1 Gate Checklist

Phase 0 (smoke tests) and Phase 1 (data infrastructure) absorb the new machinery. Every gate below is numeric and automatically checkable; the GO trigger for the build is that all Phase 0 gates pass and Phase 1 lands its dataset with every audit green. No GPU-week-scale commitment begins before that conjunction.

Table 8: Revised Phase 0 and Phase 1 gates

Phase	Gate	Threshold	Verifies
P0	Looped forward-backward throughput on T4 (hand-rolled loop, batch 16 x 600)	at least 1,500 tok/s single T4; 2,700 tok/s on 2x T4	Feasibility math of Sections 7 and 9
P0	Full r=3 probability-table build for a 30,000-bar sample	at most 2 hours on one T4; resume works after kill	Flaw 7 architecture
P0	Stage A-prime pilot (5M tokens)	NLL within 5 percent of r=1 baseline at pilot scale; lambdas move off zero	Realignment recipe trains stably
P0	Scale-invariance unit test on synthetic panel	100 percent byte-identical under a in {0.25, 0.5, 2, 4}	Flaw 1 contract is testable
P1	Cross-venue data audit (Dukascopy vs HistData, XAU + XAG + EUR + JPY)	Divergence quantified and under registered tolerance; UTC normalization verified	Data foundation incl. cross-asset panel
P1	Adversarial validation harness on train/test boundaries	AUC at or below 0.55 at every boundary	Flaw 1 gate L2
P1	Barrier-matrix labeler + bootstrap soft labels (B = 200) on CPU kernel	All cells labeled; label distributions documented per regime block	Flaws 3, 4 label schema
P1	Matched-cohort index built	at least 70 percent of anchors matched within caliper; median distance reported	Flaw 2 mechanism one
P1	M1/H1 shape codebooks fitted (training folds only) and versioned	Reconstruction sanity check; schema hash registered	Flaw 6 sequence tokens; rule SC-7
P1	aurum-data-v1 dataset published with audits green	All P1 gates above green in the build log	GO trigger for Phase 2 (baseline decider)

Phase 2 onward inherits the original roadmap unchanged except where Table 7 rewrites the internals: the baseline decider of Phase 2 now trains the joint heads at r = 1, the looped variant of Phase 3 is the Stage A-prime plus Stage B pipeline, and the Phase 4 backtest consumes probability tables. The first report's statistical release gates - Deflated Sharpe Ratio at or above 0.95, probability of backtest overfitting below 0.5, every trial counted, single touch of the test window - remain the authority on whether anything ships.

12. Residual Risks and Open Questions

Fixing seven flaws does not make the project safe; it makes it internally consistent. Six residual risks remain, deliberately stated without mitigation theater. First, the block bootstrap soft labels preserve short-lag dependence but not long-horizon regime persistence, which is why the hard-vs-soft A/B is a registered gate rather than an assumption. Second, matched-

cohort overlap can fail exactly when it is most wanted - in violent regimes, the flipped tag may not co-occur with comparable covariates, and the positivity audit will shrink the augmentation precisely then. Third, the fold-count honesty of Section 9 means the walk-forward evidence base is thinner than a full CPCV would give; the Deflated Sharpe and PBO machinery is designed for exactly this situation, but no statistic conjures information that six fewer refits did not produce. Fourth, T4 is fp16-only and Turing-old: the realignment recipe's stability on fp16 (loss scaling, no bf16 fallback) is a Phase 0 measurement, not a given. Fifth, the negative results in the looped-transformer literature still apply in full - looping may simply add nothing at 0.6B scale on this task, in which case the r=1 model ships and the loop remains a published negative ablation. Sixth, the Qwen3.5-0.8B alternate backbone remains unverified for Unsloth-style tooling, and the Jev reference video the owner mentioned has still not arrived; neither blocks the build.

One open question is flagged for the owner's judgment rather than resolved here: whether the management-pass exit head (Section 5.3) should be in the MVP or deferred. The computed budget accommodates either; the recommendation is to ship the deterministic derived exits first and let the Phase 4 ablation decide whether the learned head earns its second pass.

13. References

Sources are grouped by load. Entries marked with an arXiv identifier or URL were verified in this pass or in the two prior AURUM research phases; the prior reports' 180-source evidence base is referenced as a unit where its findings are reused. Confidence conventions follow Section 2.

13.1 Labeling, calibration, and distributional methods

1. Lopez de Prado, M. Advances in Financial Machine Learning. Wiley, 2018. (Triple-barrier method and meta-labeling, ch. 3; fractionally differentiated features, ch. 5; purged CV and the Sharpe statistics with Bailey.)
2. mlfinpy documentation, Data Labelling: Triple-Barrier Method and Meta-Labeling. mlfinpy.readthedocs.io (meta-labeling quotation and F1 framing).
3. Politis, D. N., and Romano, J. P. The Stationary Bootstrap. Journal of the American Statistical Association, 1994.
4. Koenker, R., and Bassett, G. Regression Quantiles. Econometrica, 1978; Koenker, R. Quantile Regression (Cambridge, 2001 - pinball loss survey, 8,800+ citations).
5. Muller, R., Kornblith, S., and Hinton, G. When Does Label Smoothing Help? NeurIPS 2019, arXiv 1906.02629 (label smoothing improves calibration).
6. Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. On Calibration of Modern Neural Networks. ICML 2017, arXiv 1706.04599 (temperature scaling).

7. Bellemare, M., Dabney, W., and Munos, R. A Distributional Perspective on Reinforcement Learning (C51). NeurIPS 2017; Dabney, W., et al. Distributional RL with Quantile Regression. arXiv 1710.10044.
8. Ruder, S. An Overview of Multi-Task Learning in Deep Neural Networks, 2017; Abdelsamie, M., et al. Deep Multi-Task Learning: A Review. Springer, 2026 (hard parameter sharing).
9. Kelly, J. A New Interpretation of Information Rate. Bell System Technical Journal, 1956 (fractional Kelly sizing, quarter-Kelly practice).
10. Sweeney, J. Maximum Adverse Excursion. Wiley, 1997; TradesViz, MFE and MAE: The Complete Guide, 2026 (excursion analysis practice and R-multiple conventions).
11. Morgan, S., and Harding, D. Matching Estimators of Causal Effects. Sociological Methods and Research, 2006; Stuart, E. Matching Methods for Causal Inference. Statistical Science, 2010; Abadie, A., and Imbens, G. Large Sample Properties of Matching Estimators. QJE, 2006.
12. Lazarev, E. purgedcv: purged and combinatorial cross-validation for financial ML. 2026, github.com/eslazarev/purged-cross-validation; Bailey, D., and Lopez de Prado, M. The Deflated Sharpe Ratio (JPM 2014) and The Probability of Backtest Overfitting (JCF 2016).
13. Kaggle community, Adversarial Validation (general/488194) and production drift-detection practice (apxml.com course chapter), as applied to train/test shift detection.

13.2 Looped and recursive transformer architecture

1. Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., and Schuster, T. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. ICLR 2025, arXiv 2410.20672 (multi-LoRA per loop iteration, SVD initialization +6.5 points, 60B-token continued training + distillation, 58.4 vs 58.6 recovery).
2. Geiping, J., et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach (Huginn). arXiv 2502.05171 (prelude/core/coda, input re-injection, 800B tokens).
3. Park, et al. LoopUS: Recasting Pretrained LLMs into Looped Latent Refinement Models. arXiv 2605.11011 (softplus gate, random deep supervision, 3B-token conversion).
4. McLeish, et al. Teaching Pretrained Language Models to Think Deeper with Retrofitted Recurrence. arXiv 2511.07384 (25-52B-token surgery).

5. Zhu, et al. Scaling Latent Reasoning via Looped Language Models (Ouro). arXiv 2510.25741 (7.7T-token full-stack looping, per-step loss).
6. Inner Loop Inference, arXiv 2602.14759 (zero-training looping of frozen models); loop-position retention study, arXiv 2610.19768 (99.8 percent retention with random loop supervision).
7. Bachlechner, T., et al. ReZero is All You Need. arXiv 2003.04887 (zero-initialized learned residual gates); Bai, S., Kolter, J. Z., and Koltun, V. Deep Equilibrium Models. NeurIPS 2019, arXiv 1909.01377 (fixed-point stability).
8. Residual-stream norm growth measurements (GPT-2 XL, ~1.045x per layer), LessWrong, 2023; An Analysis of Residual-Stream Geometry Across Depth, arXiv 2607.18348, 2026; Scaling Adaptive Depth with Norm-Agnostic Residual, 2026.
9. Qwen3-0.6B model card (28 layers, GQA 16Q/8KV, 32k context, Apache-2.0) and Qwen3 Technical Report. huggingface.co/Qwen/Qwen3-0.6B.

13.3 Market evidence, data, and platforms

1. Erb, C., and Harvey, C. The Golden Dilemma. Financial Analysts Journal, 2013, SSRN 2078535 (gold and real interest rates; the golden constant).
2. A Study on the Dynamic Relationship Between Gold Prices and the US Dollar Index. SSRN 6442031, 2026; He, Q., et al. Nonlinear Dynamics of Gold and the Dollar (regime-dependent inverse relationship).
3. Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on tabular data? NeurIPS 2022 Datasets and Benchmarks, arXiv 2207.08815.
4. Nie, Y., et al. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers (PatchTST). ICLR 2023, arXiv 2211.14730.
5. Jane Street Real-Time Market Data Forecasting (Kaggle, 2024-25): final leaderboard; second-place 17-model ensemble with online learning (kaggle.com/discussion/742687 and reproduction); eighth-place writeup (transformers did not work). Optiver Trading at the Close (2023-24): first-place writeup by hyd (CatBoost + GRU + Transformer blend).
6. The Alpha Illusion: An Audit of End-to-End LLM Trading Claims. arXiv 2605.16895 (P1-P6 screening protocol, adopted in the first AURUM report).
7. Kaggle notebooks documentation and staff discussion 361104 (2 x T4, 15.4 GB, 30 GPU-hours/week, 12-hour sessions, P100 retired September 2026); Unsloth T4 benchmarks (huggingface.co/blog/unsloth-trl); Microsoft, Scaling up PyTorch inference with ONNX Runtime (2.88x fp16 throughput), 2022; Hugging Face batched-inference documentation and tutorial.

8. Free data sources verified in the first research phase: Dukascopy historical feed; HistData.com (66 instruments including XAUUSD and XAGUSD M1); FRED series DGS10, T10YIE, VIXCLS, DTWEXBGS, SP500, DCOILWTICO, GOLDPMGBD228NLBM; yfinance (DX-Y.NYB, SI=F); federalreserve.gov FOMC calendar; vectorbt 1.2.0; NautilusTrader 1.231.0; Chronos tokenization arXiv 2403.07815; LLMTime digit tokenization arXiv 2310.07820.
9. ByteDance DeerFlow repository and its deep-research skill (github.com/bytedance/deer-flow), the methodology reference for this pass and both prior AURUM research reports (Project AURUM Deep Research Report; AURUM Decider Research Report and Build Plan, v1.0, 6 November 2026).